

Java 10 — Local Variable Type Inference (var)

Aspect	Details
Java Version	Java 10
Year	2018
Feature	Local Variable Type Inference
Keyword	var
UniqueClassName	VarTypeInference
Purpose	Allows the compiler to infer the type of a local variable from its initializer, reducing unnecessary type declarations.
Introduced by	JEP 286
Important point	var is not a dynamic type . Java remains statically typed; the compiler determines the variable's actual type at compile time.
Where it can be used	Local variables with initializers, enhanced for variables, traditional for-loop variables, and local variables declared in try/catch.
Where it cannot be used	Fields, method parameters, method return types, and variables without an initializer.

1. Basic Example

Traditional Java	Using var	Inferred type
String name = "Anurag";	var name = "Anurag";	String
int count = 100;	var count = 100;	int
List<String> names = new ArrayList<>();	var names = new ArrayList<String>();	ArrayList<String>
Map<String, Integer> scores = new HashMap<>();	var scores = new HashMap<String, Integer>();	HashMap<String, Integer>
Employee employee = new Employee();	var employee = new Employee();	Employee

Important: The declared type is inferred from the **initializer**, not from what you may conceptually intend.

For example:

```
var list = new ArrayList<String>();
```

The type of list is `ArrayList<String>`, not `List<String>`.

2. Advantages

Advantage	Explanation	Example
Less boilerplate	Eliminates repetitive type declarations.	<code>var employees = new ArrayList<Employee>();</code>
Improves readability	Can make complex generic declarations easier to read.	<code>var employeeMap = new HashMap<Long, Employee>();</code>
Useful with complex types	Especially valuable with nested generics and long type names.	<code>var result = service.getComplexResult();</code>
Works well with generics	Reduces duplication between variable declaration and constructor.	<code>var users = new ArrayList<User>();</code>
Useful with streams	Stream pipelines often have verbose types.	<code>var activeUsers = users.stream().filter(User::isActive).toList();</code>
Maintains static typing	The compiler still knows the exact type.	<code>var count = 10; → int</code>
Compile-time type checking	Type errors are still detected during compilation.	<code>var count = 10; count = "text"; → compilation error</code>
Refactoring convenience	Changing the concrete implementation can require fewer edits in some local declarations.	<code>var service = new PaymentService();</code>
Useful in loops	Makes loop declarations shorter.	<code>for (var employee : employees) { ... }</code>

Reduces visual noise	Particularly useful when the initializer already clearly reveals the type.	<code>var customer = new Customer();</code>
-----------------------------	--	---

3. Disadvantages

Disadvantage	Explanation	Example / Problem
Can reduce readability	The type may not be obvious from the initializer.	<code>var result = service.process(data);</code>
Can hide important abstraction	<code>var</code> may expose/infer a concrete implementation rather than the abstraction you wanted readers to see.	<code>var list = new ArrayList<String>();</code>
Requires an initializer	You cannot declare an uninitialized <code>var</code> .	<code>var name;</code> ✗
Cannot be used for fields	<code>var</code> is restricted to local variable declarations.	<code>private var name;</code> ✗
Cannot be used for parameters	Method parameters need explicit types.	<code>void save(var user)</code> ✗
Cannot be used for return types	Return types must be explicitly declared.	<code>var getUser()</code> ✗
Can create unintended concrete types	The inferred type is the initializer's type.	<code>var numbers = new ArrayList<Integer>();</code>
Can make APIs less self-documenting	Explicit types communicate intent strongly.	<code>Map<String, User></code> users may be clearer than a less obvious expression
Primitive/reference differences can be less visible	<code>var</code> can hide whether a value is primitive or reference type.	<code>var x = 10;</code> → <code>int</code>
Poor choice when type conveys business meaning	Explicit types may communicate the design better.	<code>Money salary = ...</code> is often clearer than <code>var salary = ...</code>

4. When to Use var

Situation	Use var?	Example	Reason
Type is obvious from constructor	✓ Yes	<code>var customer = new Customer();</code>	Type is immediately apparent.
Long generic declaration	✓ Yes	<code>var cache = new HashMap<Long, List<Order>>();</code>	Removes repetitive syntax.
Local collection	✓ Usually	<code>var employees = new ArrayList<Employee>();</code>	Readable and concise.
Enhanced for loop	✓ Yes	<code>for (var employee : employees)</code>	Convenient and readable.
Traditional for loop	✓ Yes	<code>for (var i = 0; i < 10; i++)</code>	Type <code>int</code> is obvious.
Try-with-resources	✓ Often	<code>try (var stream = Files.lines(path))</code>	Type is evident from the factory method.
Stream result	✓ Often	<code>var activeUsers = users.stream().filter(...).toList();</code>	Avoids verbose intermediate types.
Complex library/API type	✓ Often	<code>var response = httpClient.send(request, handler);</code>	Type can be cumbersome.
Type obvious from method name	✓ Often	<code>var customer = findCustomer(id);</code>	Method name provides context.
Type is not obvious	✗ No	<code>var result = process(input);</code>	Reader may need IDE/tooling to determine the type.
Type communicates important business meaning	✗ Usually	<code>Money salary = calculateSalary();</code>	Explicit type documents intent.
Programming to an abstraction	□ Consider carefully	<code>List<User> users = ...</code>	Explicit abstraction may be preferable.

5. When Not to Use var

Scenario	Prefer	Why
Type is important for understanding the code	Explicit type	Improves documentation.
Method returns an unclear type	Explicit type	Makes the result immediately understandable.
Interface/abstraction is important	Interface type	Communicates design intent.
Public API	Explicit type	var cannot be used for parameters/returns anyway.
Variable has a business-specific meaning	Explicit type	Makes domain semantics clearer.
Initializer doesn't reveal type	Explicit type	Prevents readers from having to inspect the method.
You want to deliberately program against an interface	Explicit interface type	Avoids coupling the reader's understanding to implementation.
Team coding standards discourage var	Follow team standard	Consistency is more valuable than personal preference.

6. Real-Time Use Cases

Use Case	Example	Why var is useful
Creating domain objects	<code>var order = new Order();</code>	Type is obvious.
Collections	<code>var users = new ArrayList<User>();</code>	Reduces duplicated generic syntax.
Maps	<code>var ordersByCustomer = new HashMap<Long, List<Order>>();</code>	Complex generic type becomes easier to read.
Stream processing	<code>var activeUsers = users.stream().filter(User::isActive).toList();</code>	Avoids verbose result declaration.
Database results	<code>var customers =</code>	Useful when method name clearly

	<code>customerRepository.findActiveCustomers();</code>	identifies the result.
File processing	<code>var path = Paths.get("/data/orders.csv");</code>	Type is obvious from <code>Paths.get</code> .
HTTP client code	<code>var request = HttpRequest.newBuilder(uri).GET().build();</code>	Constructor/factory makes type clear.
Configuration objects	<code>var config = new ApplicationConfig();</code>	Obvious concrete type.
Loop processing	<code>for (var order : orders)</code>	Removes repetitive type declaration.
Temporary calculation	<code>var total = price * quantity;</code>	Type is generally obvious from the expression.

7. Real-Time Example — E-Commerce

Without var

```
List<Order> pendingOrders =
    orderService.findPendingOrders();

Map<Long, List<Order>> ordersByCustomer =
    new HashMap<Long, List<Order>>();

for (Order order : pendingOrders) {
    // process order
}
```

With var

```
var pendingOrders = orderService.findPendingOrders();

var ordersByCustomer = new HashMap<Long, List<Order>>();

for (var order : pendingOrders) {
    // process order
}
```

Best practice: `var` is particularly useful when the type is obvious and doesn't add meaningful information.

8. Important Difference: var vs Dynamic Typing

var in Java	Dynamic typing
Compile-time type inference	Runtime type determination
Java remains statically typed	Usually associated with dynamically typed languages
Variable's type cannot arbitrarily change	Variable can often hold different types
Compiler performs type checking	Many type checks occur at runtime
var x = 10; → int	Conceptually, a dynamic variable could later hold a string
x = "Hello"; ✗	Often allowed in dynamically typed languages
Introduced in Java 10	Not a conversion of Java into a dynamic language

So:

```
var value = 10;  
value = 20;    // ✓  
value = "Hello"; // ✗
```

The compiler has inferred:

```
int value = 10;
```

9. var vs Explicit Type

Factor	Explicit Type	var
Readability	Excellent when type is meaningful	Excellent when type is obvious
Boilerplate	More	Less
Type visibility	Immediate	Sometimes hidden
Static typing	✓	✓

Compile-time safety	✓	✓
Fields	✓	✗
Parameters	✓	✗
Return types	✓	✗
Local variables	✓	✓
Generic-heavy code	Can become verbose	Often cleaner
API design	Preferred	Not applicable
Business/domain semantics	Often better	Can obscure intent
Refactoring	May require more declaration changes	Often less repetitive
Runtime performance	No inherent difference	No inherent difference

10. Alternate Features to var

var doesn't have a single direct replacement because it primarily solves **local variable declaration verbosity**. The best alternative depends on what you are trying to accomplish.

Alternative	Java Version	What it provides	When preferable to var	Example
Explicit variable type	Java 1.0+	Directly declares the intended type	When type is important for readability	List<User> users = ...;
Diamond operator <>	Java 7	Type inference for generic object creation	When the main problem is repeated generic type arguments	List<User> users = new ArrayList<>();
var	Java 10	Local variable type inference	When the type is obvious from the initializer	var users = new ArrayList<User>();

val-style libraries	Third-party	Often provide immutable/inferred locals	When a project deliberately adopts such conventions	Library-dependent
IDE type inference/code completion	Tooling	Shows inferred types without changing source	When explicit declarations improve source readability	IDE displays actual inferred type
Factory methods	Various	Hide construction complexity behind descriptive methods	When construction type details are implementation concerns	var customer = Customer.create(...);
Builder pattern	Various	Simplifies complex object creation	When constructors/builders are complicated	var request = Request.builder()...build();

Most important alternative: Diamond Operator

There is a useful relationship:

```
List<String> names = new ArrayList<>();
```

Here, `<>` infers the generic type arguments.

Whereas:

```
var names = new ArrayList<String>();
```

Here, **var** infers the **variable's type**.

They solve related but different problems.

11. Best-Practice Decision Table

Question	If Yes	Recommendation
Is the type obvious from the initializer?	Yes	✓ Use var

Is the declaration unnecessarily repetitive?	Yes	✓ Use var
Is the generic type very long?	Yes	✓ Consider var
Does the explicit type communicate important design intent?	Yes	✗ Prefer explicit type
Is the initializer a method with an unclear return type?	Yes	✗ Prefer explicit type
Are you deliberately coding against an interface?	Yes	✗ Prefer the interface type
Is the variable only used locally and briefly?	Yes	✓ var can be appropriate
Will a future reader need to know the exact type immediately?	Yes	□ Prefer explicit type
Are you using var merely to reduce every possible character?	Yes	✗ Don't use it mechanically
Does your team have a consistent var convention?	Yes	✓ Follow the project convention

12. Recommended Rule for VarTypeInference

Rule	Recommendation
Primary rule	Use var when the initializer makes the type immediately obvious.
Do not use it mechanically	var is a readability tool, not a requirement.
Prefer explicit abstractions	Use List, Map, interfaces, or domain types when that abstraction communicates intent.
Use for obvious constructors	var customer = new Customer(); is a strong use case.

Use for complex generics	<code>var cache = new HashMap<Long, List<Order>>();</code> is often cleaner.
Avoid mysterious expressions	<code>var result = process(data);</code> may hide too much information.
Remember static typing	<code>var</code> does not make Java dynamically typed.
Don't expect runtime performance gains	<code>var</code> is primarily a source-code readability/maintainability feature , not a performance optimization.

Bottom line

var is best viewed as a readability and boilerplate-reduction feature. It is excellent when the initializer already tells the reader what the type is; it is less appropriate when the type itself carries important information.

Good:

```
var customer = new Customer();
var orders = orderRepository.findPendingOrders();
var cache = new HashMap<Long, List<Order>>();
```

Less desirable:

```
var result = process(data);
var value = getSomething();
```

In the latter cases, an explicit type can make the code substantially easier to understand.

Java 10 — AppCDS Improvements

Aspect	Details
Java Version	Java 10
Year	2018
Feature	AppCDS Improvements
UniqueClassName	AppCDS
Full Name	Application Class-Data Sharing

Primary purpose	Improve JVM startup time and reduce memory footprint by sharing class metadata between JVM processes.
Underlying technology	Class Data Sharing (CDS)
Java 10 improvement	AppCDS was enhanced to allow application classes to be included in the shared archive, rather than being limited mainly to bootstrap classes.
Main benefit	Faster startup and potentially lower memory usage when many JVM instances use the same application classes.
Typical environment	Servers, microservices, containers, command-line applications, cloud deployments.
Performance category	Startup performance + memory efficiency
Runtime behavior	Classes can be loaded from a pre-generated shared archive instead of being processed independently by every JVM.

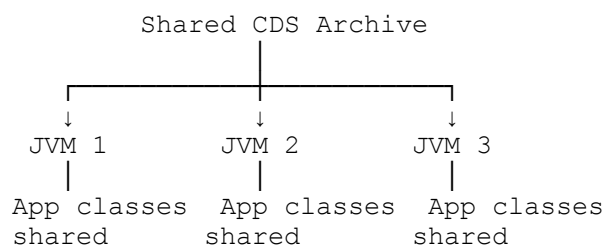
1. What is AppCDS?

Application Class-Data Sharing (AppCDS) extends Java's Class Data Sharing mechanism so that classes belonging to an application can also be placed into a shared archive.

Without AppCDS:

```
JVM 1 → loads application classes → creates class metadata
JVM 2 → loads application classes → creates class metadata
JVM 3 → loads application classes → creates class metadata
```

With AppCDS:



The idea is particularly valuable when **many JVM processes run the same application**.

2. Advantages of AppCDS

Advantage	Explanation	Practical Benefit
Faster JVM startup	Pre-archived class metadata can reduce class-loading work.	Applications become ready sooner.
Reduced memory footprint	Multiple JVMs can share read-only class metadata/pages.	Useful when many JVMs run on the same machine.
Better microservice startup	Small services can benefit when frequently started.	Faster container/service deployment.
Useful for CLI applications	Short-lived Java applications can spend a significant portion of their lifetime starting.	Lower startup overhead.
Improves application scalability	Multiple instances can potentially share archived class data.	More efficient use of host memory.
Works with existing Java applications	It does not require rewriting application business logic.	Relatively low application-code impact.
Useful in container environments	Repeatedly starting the same application makes shared class data valuable.	Better startup characteristics.
Can complement other JVM optimizations	It targets startup/class-loading overhead rather than application algorithms.	Can be combined with normal JVM tuning.
No source-code changes required	AppCDS configuration is primarily a JVM/archive-generation concern.	Existing applications can benefit.

3. Disadvantages of AppCDS

Disadvantage	Explanation	Impact
--------------	-------------	--------

Additional deployment complexity	You must create, maintain and use CDS archives.	More operational work.
Archive must match the application/runtime appropriately	Changes to classes, classpath/module configuration, or runtime environment can make an archive unsuitable or less useful.	Archives may need regeneration.
Not primarily a runtime throughput optimization	AppCDS mainly targets startup and memory sharing.	It won't automatically make business logic faster.
Less useful for a single long-running JVM	Memory-sharing benefits are more valuable when multiple JVMs share the same classes.	Limited benefit in some deployments.
Archive generation adds build/deployment steps	Production pipelines may need an archive-generation stage.	CI/CD becomes more involved.
Potential maintenance overhead	New application versions may require new archives.	More artifacts to manage.
Benefits vary by workload	Applications with little startup/class-loading overhead may see modest improvement.	Benchmarking is important.
Class compatibility considerations	Application/classpath changes can affect archive usability.	Configuration must be controlled.
Debugging/deployment can be slightly more complicated	JVM startup configuration includes CDS-related options.	Operators need to understand the mechanism.

4. When to Use AppCDS

Situation	Use AppCDS?	Reason
Many instances of the same Java application	✓ Yes	Shared class data can reduce aggregate memory usage.

Microservices	✓ Often	Many JVM instances may start independently.
Containerized Java applications	✓ Often	Faster startup can improve deployment/scaling behavior.
Serverless-style Java workloads	✓ Potentially	Startup time is important.
Frequently restarted applications	✓ Yes	Startup improvements are repeatedly realized.
Short-lived Java CLI programs	✓ Potentially	Startup overhead represents a larger fraction of total execution time.
Large applications with many classes	✓ Potentially	Class-loading/metadata overhead can be significant.
Application deployed to a shared host with many JVMs	✓ Yes	Memory sharing can be particularly valuable.
Stable production application	✓ Often	A stable classpath makes archive management easier.
High-density Java deployment	✓ Yes	Memory savings can allow more instances per host.

5. When Not to Use AppCDS

Situation	Use AppCDS?	Reason
One long-running JVM	☐ Usually low priority	Startup happens rarely, so startup savings may have little business value.
Tiny application with negligible startup time	✗ Usually no	Benefit may not justify configuration complexity.
Application changes constantly	☐ Consider carefully	Archives may need frequent regeneration.
Highly dynamic deployment/classpath	☐ Consider carefully	Maintaining an effective archive can be difficult.
Application performance	✗ No	AppCDS doesn't directly optimize

problem is CPU-bound business logic		the business algorithm.
Database is the bottleneck	✗ No	AppCDS won't solve database latency.
Network latency is the bottleneck	✗ No	CDS doesn't reduce network latency.
Memory problem is caused primarily by application objects	✗ Not the main solution	AppCDS targets class metadata sharing, not arbitrary heap objects.
Rarely restarted batch process	☐ Usually low priority	Startup savings may be insignificant compared with processing time.

6. Real-Time Use Case — Microservices

Suppose a company runs:

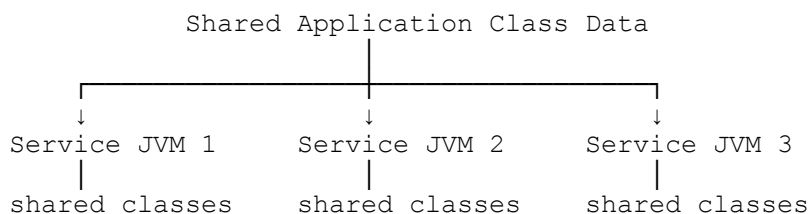
Customer Service → JVM
 Payment Service → JVM
 Order Service → JVM
 Inventory Service → JVM
 Notification Service → JVM

Each service uses a large number of common Java/framework classes.

Without effective class-data sharing:

JVM 1 → loads common classes
 JVM 2 → loads common classes
 JVM 3 → loads common classes
 JVM 4 → loads common classes
 JVM 5 → loads common classes

With AppCDS:



Benefit

This can help when a host runs many copies of similar Java applications, particularly where **startup speed and memory density** matter.

7. Real-Time Use Case — Container Scaling

Imagine a Java REST service deployed in Kubernetes.

During normal traffic:

3 Java containers

During a traffic spike:

3 → 10 → 30 Java containers

Every new container starts a JVM and loads application classes.

AppCDS can help reduce the startup/class-loading work associated with repeatedly launching the same application.

This can be useful for:

- autoscaling
 - rolling deployments
 - blue/green deployments
 - rapid replacement of failed containers
 - burst workloads
-

8. Real-Time Use Case — CLI Application

Consider a Java command-line utility:

```
java -jar report-generator.jar
```

Suppose the program:

1. Starts the JVM.
2. Loads many framework/application classes.
3. Performs a relatively small operation.
4. Exits.

If startup takes a noticeable portion of total execution time, reducing startup overhead can have a meaningful effect.

For a long-running server that executes for days, the same startup improvement may be much less significant.

9. Conceptual AppCDS Workflow

Step	Activity
1. Run application	Execute the application in a representative way.
2. Collect class information	Identify classes loaded during the representative execution.
3. Create archive	Generate an AppCDS archive containing eligible application class data.
4. Deploy archive	Package/deploy the archive with the application.
5. Start JVM using archive	JVM uses the archived class data when possible.
6. Measure	Compare startup time and memory usage against the non-AppCDS configuration.

A simplified conceptual command sequence can look like:

```
java -Xlog:class+load=info -jar app.jar
```

followed by archive-related CDS tooling/options appropriate to the particular JDK release.

Important: Exact AppCDS command-line options and workflow have evolved across JDK releases, so production deployment should use the documentation for the specific JDK version being deployed.

10. AppCDS vs Normal CDS

Feature	Traditional CDS	AppCDS
Full name	Class Data Sharing	Application Class-Data Sharing
Primary goal	Share JVM class metadata	Extend sharing to application

		classes
Application classes	Historically limited	✓ Supported
JDK/platform classes	✓	✓
Startup improvement	✓	✓
Memory sharing	✓	✓
Useful for multiple JVMs	✓	✓
Application-specific archive	Limited	✓
Java 10 significance	Existing CDS mechanism	Major expansion of CDS to application classes

11. Alternate Features to AppCDS

Alternative	Java Version / Technology	Main Purpose	When It May Be Better
CDS	Java 5-era technology, substantially evolved later	Share class metadata	When application-class archiving isn't necessary.
Default CDS archive	Modern JDKs	Share standard JDK class data	Simple deployments where default sharing is sufficient.
Dynamic CDS	Modern JDKs	Generate/use CDS archives more conveniently at runtime	Modern deployments that want less manual archive management.
AOT compilation	Various JVM technologies	Compile code ahead of execution	When reducing startup and warm-up is more important than only sharing class metadata.
GraalVM Native Image	GraalVM	Compile Java application into a native executable	Excellent for very fast startup and low memory environments, with compatibility/trade-off

			considerations.
CRaC	Modern JVM ecosystem	Restore a pre-initialized JVM/application state	When extremely fast startup/resume is required.
JVM warm-up/tuning	JVM-wide	Optimize JIT/runtime behavior	When the real problem is warm-up or runtime throughput rather than class loading.
Application/framework optimization	Any	Reduce initialization work	When startup is dominated by framework initialization, configuration, database connections, etc.

12. AppCDS vs Native Image

Factor	AppCDS	GraalVM Native Image
Basic approach	Share archived JVM class data	Ahead-of-time native compilation
JVM required	✓ Yes	✗ Typically no traditional JVM at runtime
Startup improvement	✓	✓ ✓ Often much stronger
Memory efficiency	✓ Can help	✓ Often strong
Java compatibility	Generally high	Can require compatibility/configuration adjustments
Existing JVM model	Preserved	Replaced by native executable model
Deployment complexity	Moderate	Moderate/high depending on application
Best for	JVM applications needing better startup/memory characteristics	Applications where very fast startup and native deployment are major priorities

Runtime JIT	✓	Typically no traditional HotSpot JIT
Migration effort	Low/moderate	Potentially higher
Primary category	JVM optimization	AOT/native compilation

13. AppCDS vs Other Startup Optimizations

Feature	Primary Target	Startup	Memory	Runtime Throughput
AppCDS	Class metadata sharing	★ ★ ★	★ ★ ★	—
JIT tuning	Runtime compilation	★ ★	★ ★	★ ★ ★
GraalVM Native Image	AOT compilation	★ ★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★
CRaC	Checkpoint/restore	★ ★ ★ ★ ★	★ ★	★ ★ ★
Lazy initialization	Application initialization	★ ★ ★	★ ★ ★	★ ★
Framework optimization	Startup initialization	★ ★ ★ ★	★ ★ ★	★ ★

Ratings are qualitative and workload-dependent, not guaranteed benchmark results.

14. Best-Practice Decision Table

Question	Recommendation
Do you run many copies of the same JVM application?	Strong AppCDS candidate
Is startup time important?	Consider AppCDS
Is memory density important?	Consider AppCDS
Do you frequently start/restart JVMs?	Consider AppCDS

Is the application stable between deployments?	AppCDS becomes easier to manage
Is your application continuously changing its classpath?	Benchmark carefully before adopting
Is your bottleneck database/network/business logic?	AppCDS is unlikely to solve it
Do you need ultra-fast startup?	Also evaluate Native Image or CRaC
Is your JVM long-running for weeks/months?	AppCDS startup benefit may be relatively small
Do you want zero source-code changes?	AppCDS is attractive
Are you optimizing a high-density container platform?	AppCDS is worth benchmarking

15. Final Summary

Category	AppCDS Assessment
Main purpose	Share application class data between JVM processes
Biggest advantage	Faster startup + potentially lower memory usage
Biggest disadvantage	Archive generation and maintenance complexity
Best environment	Microservices, containers, high-density JVM deployments
Best workload	Frequently started/restarted applications
Poor fit	Single, long-running JVM where startup is insignificant
Source-code changes	Generally not required
Runtime performance	Not primarily a throughput optimization
Main alternative	CDS / modern dynamic CDS
More aggressive startup alternative	GraalVM Native Image or CRaC
Recommended approach	Benchmark startup time, RSS/memory usage, and deployment

	complexity before adopting
--	----------------------------

Key takeaway: Java 10's AppCDS improvement made CDS substantially more useful for real applications by allowing application classes to participate in class-data sharing. It is particularly valuable when many JVM instances of the same application start on the same host, making startup time and memory efficiency important.